

# Algorithm Design Paradigms

## Handout on Recurrence and Induction

20 January 2026

### §1 Introduction

I originally was planning to teach this class via ‘Mathematical Circles’ but then I decided to not insult the intelligence of everyone here by using a book that was fit for you all a decade ago and simply start with more interesting stuff.

A lot of the algorithms we will look at will talk about multiple steps and calling certain sub-routines again and again.

#### **i Remark**

The hardest part about induction is remembering to use it.

Here are some situations where induction is especially helpful:

- Any problem involving a recurrence relationship or a sequence.
- Any problem involving counting, because you can often reduce a counting problem to a recurrence relationship.
- Any problem involving games. If you can guess exactly what the winning positions are, it's probably easy to prove your guess with induction.
- Proving something exists. You can do this by inductively assuming something smaller exists, and then extending it to what you want.

#### **? Problem**

A number of red points and blue points are drawn in a unit square with the following properties:

- The top-left and top-right corners are red points.
- The bottom-left and bottom-right corners are blue points.
- No three points are collinear.

Find an algorithm to draw red segments between red points and blue segments between blue points in such a way that: all the red points are connected to each other, all the blue points are connected to each other, and no two segments cross.

This problem is just trying to scare you. If I was the setter, I would have written arbitrary quadrilateral. Does that give you all an hint?

We make the modified claim

**Claim 1.1.** Consider a triangle where two vertices are one colour and already connected by a segment, and the third vertex is the other colour. Then it is possible to draw segments inside the triangle connecting all points of the same colour.

We will prove this by induction on the number of points inside the triangle.

(B) If the triangle has no points inside, it's obvious.

(S) Otherwise consider a triangle  $ABC$ , and assume without loss of generality that  $AB$  is blue and  $C$  is red. If  $ABC$  has no internal red points, we can just connect everything with blue segments and we're done.

Otherwise,  $ABC$  has some internal blue point  $P$ . We will colour  $CP$  red. Each triangle  $ABP$ ,  $BCP$ ,  $CAP$  has fewer internal points than  $ABC$  (since  $P$  is not inside any of them), so by the inductive hypothesis, it is possible to draw segments inside each triangle  $ABP$ ,  $BCP$ ,  $CAP$  to connect all points of the same colour. But then every red point in  $ABC$  is connected to  $P$ , and every blue point is connected to either  $A$  or  $B$  and those two points are directly connected to each other. Thus, we can also connect  $ABC$  and the inductive result follows.

### ? Problem

Let  $n$  be a positive integer. We have  $n^2$  different mithai in  $n$  different flavors, not necessarily  $n$  of each flavor.

Create a scheme to distribute  $n$  sweets to  $n$  kids with  $n$  sweets per kid, such that each kid has at most 2 different flavors.

See, reasoning about this problem at a global level seems too hard. Let's think of it as a  $n$  step process where in the  $i^{\text{th}}$  step we assign  $n$  sweets to  $i^{\text{th}}$  kid.

**? Problem**

Prove the correctness of the given algorithm that outputs a value  $v \in [(1 - \varepsilon) \log k, (1 + \varepsilon) \log k]$  given  $k > 1$  and  $\varepsilon > 0$ .

```
def logTay(k, eps):
    if k - 1 < eps:
        return k - 1
    else:
        return 2 * logTay(sqrt(k), eps)
```

We are no longer dealing with naturals. So what do we do?

*Proof.* We induct on number of recursive calls made to logTay.

□

This gives me a nice leeway into

## §2 Recursion

Everyone who already codes might feel like recursion is a rather basic and fundamental idea. But it almost never made it to Computers.

When Algol was being developed, most of CS community believed the idea of a function calling itself was stupid and risky. Famously, when recurrence was just discussed at a conference, the speaker was interrupted and some academic I can't recall the name of remarked "Guys we are grown men. Unlike boys we can't talk about imaginary play things for our merriment."

So why was recursion eventually included? While that is a rather long and interesting story and beyond the scope of this tutorial (link below), it features a name you'll hear again in a few days, Edger Dijkstra.

Anyways, let's channel our inner 1950's academics and ask, "why care about recursion?"

**? Problem**

Guess Who is a popular board game. An abstract version of this is:

- You and your opponent choose a number  $1 \leq b_1, b_2 \leq N$ , secret from each other.
- Every turn you ask each other a subset of numbers between  $1 - N$  reply yes if the number is in the subset and no otherwise.
- The first player to ask a singleton set and receive a positive response, wins.

How many of you have watched the Mark Rober video about this? A simple idea looks like quarrying a random half of the numbers remaining every turn. (the coders in the house might be shouting binary search). Where is the recursion even needed?

Um, that is bat shit and is not a good strategy. Say your opponent (going first) quarried a size 15 set in a game of 20 and got a 'no'. Now, they have a search space of size 5 while you are still left with 20.

Let me make you another silly game.

### ? Problem

Calvin and Hobbes wish to divide 25 coins, of denominations 1, 2, 3, ..., 25 kopeks. In each move, one of them chooses a coin, and the other player decides who must take this coin. Calvin makes the initial choice of a coin, and in subsequent moves, the choice is made by the player having more kopeks at the time. In the event that there is a tie, the choice is made by the same player in the preceding move. After all the coins have been taken, the player with more kopeks wins. Which player has a winning strategy?

Can we try to encode everything about the game in a single state? For example a chess board state and whose turn tells us everything we want to know, can we do the same here?

If you think about it, every game without chance or hidden information, for example Chess; can have a representation and then be traversed in this same fashion. That's exactly how StockFish works.

The first example is called a Stochastic Game while the second is called a Combinatorial Game. While the complete analysis of former needs

"Ok, but these are games. We are supposed to do CS, not play games."

### ? Problem

Given an input  $a$  and a prime modulus  $p > a$ , output  $a^{-1} \bmod p$  where  $a^{-1} \bmod p \in \{1, 2, \dots, p - 1\}$  such that  $a * a^{-1} \bmod p = 1$

But we don't even need to go into cryptographic functions. A very common function on your phone uses this.

### ? Problem

Given  $b \in \mathbb{R}$  and  $e \in \mathbb{Z}^+$ , calculate  $b^e$  with as little number of multiplications as possible.

If anyone thinks the recursive answer is

```
slow b 0 = 1
slow b e = b * slow (b (e-1))
```

then they are sadly mistaken. This is very slow. What about

If you look at it, we divided the problem into 2 smaller, equal parts and solved them. And this kind of a strategy works in a lot of places, so many places that it has a name: **Divide and Conquer**